r_tutorial_week_01

Werkcollege 1

Sebastian A. J. Lortz

15 October 2026

Table of contents

How to use this tutorial **2**

1 Your computer's filing system **2**
Your turn . . . 3

2 Opening RStudio **4**
Your turn . . . 4
2.1 The four panes . . . 4

3 Working directory **4**
Your turn . . . 5

4 R scripts **5**
Your turn . . . 6
Your turn . . . 6

5 R as a calculator **6**
Your turn . . . 7
Your turn . . . 7
Your turn . . . 8
5.1 Checking your working directory . . . 8
Your turn . . . 8

6 Objects: storing values for later **8**
Your turn . . . 10

7 Comparison operators **10**
Your turn . . . 11

8 Vectors: working with many values at once **11**
Your turn 12

9 Picking out values (indexing) **12**
Your turn 13

10 Functions **14**
Your turn 14

11 Borrowing extra tools: packages **15**
Your turn 15

12 Saving your work **16**
Your turn 16

13 Recap: what you can now do **16**

How to use this tutorial

Welcome to the R tutorial! You do **not** need any prior experience. Work through this page carefully, from top to bottom. Every short explanation is followed by a **Your turn** task to do yourself.

General tips:

- Read each part slowly and in order. Each step builds on the one before it.
- Type the code yourself instead of copying and pasting it. You learn faster this way.
- Errors are normal. Read the message and try again.
- If you are stuck, check the FAQ or ask a teacher.
- Save your work often.

---

1 Your computer's filing system

Before we start with RStudio, we discuss some important terms about your computer itself.

A **file** is a single object (a photo, a Word document, an R script). A **folder** (also called a **directory**) is a box that holds files and other folders. Folders can sit inside other folders, building a **folder structure**.

Example: a tidy folder structure

A tidy folder structure can look like this:

```
Documents
   first_year
       research_methods
[screenshot from UWP]
```

At the root of this structure is a **drive**. A drive is the storage of your computer. On a personal computer you usually have only one drive. Inside the drive, every computer has some common **base folders**, such as the **Desktop** (the screen you see when your computer starts), **Documents** (where files are usually stored), and **Downloads** (where files you download are saved).

It is important to keep this structure tidy. Work inside *Documents* and make one folder for your first-year studies. Inside that folder, make a folder for each course. Inside each course folder, you can sort your material further.

Do **not** build your folder structure inside the *Downloads* folder. Everything you download lands there automatically, so it quickly becomes messy.

Your turn

Create one folder for this whole course, so everything stays in one place.

1. Open your file explorer (*File Explorer* on Windows, *Finder* on Mac).
2. Go to your *Documents* folder.
3. If it is not there already, create a folder for your studies and name it `first_year`.
4. Inside `first_year`, create a new folder and name it `research_methods`.

Good names for files and folders

- Keep folder and file names simple: use lowercase letters, numbers, and underscores `_` instead of spaces. For example, `my data.csv` can cause problems later, but `my_data.csv` does not.
- Use short but clear names, so you know what is inside without opening the folder.
- When a folder starts to hold many different things, make subfolders to group them.

You now have a home for your course work.

---

2 Opening RStudio

R is the engine that does the computations. **RStudio** is the user interface you use to talk to that engine comfortably. You installed both before class (see the Brightspace instructions). We always open RStudio only and **never** open R itself.

Your turn

Open the **RStudio** program (not plain "R"). We work with RStudio version [...] running R version [...].

> **!** Stuck on installation?
>
> Raise your hand right away. A teacher will help you.

2.1 The four panes

RStudio's window is split into panes.

- **Editor** (top-left): where you **write** and save code (empty for now).
- **Console** (bottom-left): where R **runs** commands and shows results.
- **Environment** (top-right): a list of everything you have **stored** (empty for now).
- **Files / Plots / Packages / Help** (bottom-right): your file browser, graphs, installed packages, and help pages.

Find the *Console*. You will see a `>` symbol. That `>` means "R is ready and waiting for you."

> Getting the `>` back
>
> While programming, the `>` symbol is sometimes not shown. When this happens, R is waiting for the rest of an unfinished line. Click in the *Console* and press *Esc* (escape) on your keyboard to get the `>` symbol back.

---

3 Working directory

RStudio needs to know where to work, for example, where to look for files and data. The first thing we do when working in RStudio is to set the **working directory**, that is the folder we are working in. Because we already made a folder for this course, we choose that one.

Your turn

1. In the menu, click *Session* > *Set Working Directory* > *Choose Directory*.
2. A window opens that shows your computer's file system. Navigate to the `research_methods` folder you made and select it.
3. Click *Open*.

RStudio has now set the working directory to that folder. In the *Console*, a line appears that starts with `setwd(...)` and shows the path to your folder, for example:

> In the *Console* you now see:
>
> `> setwd("path_to_your_research_methods_folder")`

The exact path is different on every computer, so yours will look different from others.

---

4 R scripts

In this course we only work with **R scripts**, which are shown in the *Editor* pane (top left). An R script is a place where you type and save lines of code, which are then run. A script runs from top to bottom.

We work with R scripts because everything you do in RStudio is lost when you close the program. Without a script, you would have to start again from scratch every time. But if you save your R script, you can open it again next time and run the lines to build your work back up.

For this to work, keep your script tidy: only keep the lines that work and that are needed to produce your output. Remove anything you tried that did not work. This way, running the script from top to bottom always gives a clean result.

> Example: a messy and a tidy script
>
> You do not need to understand the code yet. Just look at the **shape** of the two scripts. A **messy script** still holds every try, including lines that gave an error:

```
2 + 3
2 +
2 and 3
3 x 4
3 * 4
3*4
```

A **tidy script** has a title, a short comment for each step, and only the lines that work.

```
# R tutorial - Week 1
# Name: ...
# Date: ...

# Add two numbers
2 + 3

# Multiply two numbers
3 * 4
```

Think of a recipe: it lists only the steps that worked, not the things you burnt on the first try.

This is important for **reproducibility**: you can give your tidy script to another person, and they get the same output. We never type our code straight into the console.

Your turn

1. Create a new R script: click *File* > *New File* > *R Script*.
2. Save it right away: press *Ctrl* + *S* on Windows (*Cmd* + *S* on Mac), or click *File* > *Save*.
3. RStudio asks where to save the script. Choose your `research_methods` folder and name the file `week1.R`.
4. Click *Save*. The name `week1.R` now appears on the tab at the top of the *Editor*.

In a script, we write notes with a `#` (hash) at the start of a line. A line that starts with `#` is called a **comment**. R does not run a comment as code; it skips it. Comments help you and others understand what your code does.

Your turn

Click in the first line of your new R script. Type a `#` and then a title for your script, for example: `# R tutorial - Week 1`.

---

5 R as a calculator

The fastest way to meet R is to use it as a calculator.

Your turn

Type each line below into your R script. Press *Enter* after each one, so the lines sit underneath each other.

```
2 + 3
#> [1] 5
10 - 4
#> [1] 6
6 * 7
#> [1] 42
20 / 5
#> [1] 4
```

To run a line, click somewhere in that line and:

- on Windows, hold *Ctrl* and press *Enter*;
- on Mac, hold *Cmd* and press *Enter*.

R runs the line and jumps to the next one.

Your turn

Run the four calculations above, one by one.

R prints the answer back. Ignore the `[1]` in front for now – it just means "this is the first result."

R also knows powers and square roots:

```
# 2 to the power of 3
2 ^ 3
#> [1] 8

# the square root of 9
sqrt(9)
#> [1] 3
```

Note that `sqrt()` is a function; you learn more about functions later.

Remember: anything after a `#` is a **comment**: a note for humans that R ignores. Use comments to remind yourself what code does.

Your turn

In your script, work out:

1. `144` divided by `12`.
2. `15` times `4`.

5.1 Checking your working directory

Earlier you set the working directory with the menu. You can check which folder it is with the function `getwd()` (**get** **w**orking **d**irectory):

```
# which folder is my working directory?
getwd()
```

R prints the path of the folder, for example `"C:/Users/you/Documents/first_year/research_methods"`. Your path will look different, but it should end with `research_methods`.

Your turn

1. Type `getwd()` in your script, with a comment above it, and run the line.
2. Does the path end with `research_methods`?
3. If not, set the working directory again: *Session* > *Set Working Directory* > *Choose Directory*.

---

6 Objects: storing values for later

Re-typing numbers every time is tedious. Instead we store a value in an **object** and give it a name. We do this with the **assignment arrow** `<-`, which you read as "gets."

```
age <- 19
```

Read that as: "the object `age` gets the value 19." Notice nothing was printed. The value is now stored in the environment. To see it, type the name on its own:

```
age
#> [1] 19
```

> **Shortcut for the assignment arrow**
>
> Press *Alt* and - (minus) on Windows, or *Option* and - on Mac. RStudio types `<-` with the spaces for you.

Once stored, an object can be used in calculations just like the number itself:

```
# a score on the self-efficacy scale we will collect
self_efficacy <- 32

# 32 + 8 = 40
self_efficacy + 8
#> [1] 40
```

You can also **overwrite** an object by assigning a new value to the same name:

```
# age is now 20, the old value (19) is gone
age <- 20

# print the new value
age
#> [1] 20
```

Check the *Environment* pane (top-right): `age` and `self_efficacy` now appear there. That pane is your live list of stored objects.

> **Watch out: naming rules**
>
> - Names may use letters, numbers, `.` and `_`, and must **start with a letter**.
> - No spaces. Use `self_efficacy`, not `self efficacy`.
> - Names are **case-sensitive**: `Age`, `age`, and `AGE` are three different objects.

```
# we never created "Age" with a capital A, so R complains
Age
#> Error:
#> ! object 'Age' not found
```

> ⚠ Watch out: errors
>
> In RStudio, this message appears in red. It is called an **error**: the message starts with the word `Error`. An error means R stopped: the code did not work. Here it simply means "I do not know an object called `Age`." Do not panic when you see an error: read it as a hint about what went wrong.

Your turn

Create an object called `study_hours` and give it the value 12. Then create `study_hours_per_day` equal to `study_hours / 7` and print it.

---

7 Comparison operators

Comparison operators are used to compare objects or numbers. They give back one or more **logical** values: either `TRUE` or `FALSE`.

These are the most common comparison operators:

Operator	Meaning
`==`	is equal to
`!=`	is not equal to
`>`	is greater than
`<`	is smaller than
`>=`	is greater than or equal to
`<=`	is smaller than or equal to

For example:

```
# is 10 greater than 5?
10 > 5
#> [1] TRUE

# is 10 smaller than 5?
10 < 5
#> [1] FALSE
```

```
# is 10 equal to 10?
10 == 10
#> [1] TRUE
```

> ⚠ Watch out
>
> To ask "is it equal?" you must use **two** equals signs `==`. A single `=` means something different and will not work here.

Your turn

In your script, check whether:

1. `7` is greater than `3`, and
2. `4` is equal to `5`.

Run each line and read the `TRUE` / `FALSE` answer.

---

8 Vectors: working with many values at once

In research we rarely have one number; we have many (one score per participant). A **vector** is an ordered collection of values. We build one with `c()`, which stands for combine or concatenate. Put the values inside, separated by commas.

```
scores <- c(28, 35, 31, 40, 22)
scores
#> [1] 28 35 31 40 22
```

`scores` now holds five values in one object.

A powerful feature: when you do arithmetic on a vector, it applies to **every element at once**. You do not have to do it by hand for every value.

```
# add 10 to each score
scores + 10
#> [1] 38 45 41 50 32

# double each score
```

```
scores * 2
#> [1] 56 70 62 80 44
```

Vectors do not have to be numbers. A **character** vector holds text (each piece in quotes), and a **logical** vector holds `TRUE`/`FALSE` values:

```
first_names <- c("Sanne", "Tim", "Noor", "Mees", "Lotte")
passed <- c(TRUE, FALSE, TRUE, TRUE, FALSE)
first_names
#> [1] "Sanne" "Tim"   "Noor"  "Mees"  "Lotte"
passed
#> [1]  TRUE FALSE  TRUE  TRUE FALSE
```

> ⚠ Watch out
>
> Every value needs its own comma, and text needs quotes. `c("Sanne" "Tim")` (missing comma) gives an error; so does `c(Sanne, Tim)` (missing quotes – R would look for objects named Sanne and Tim).

Your turn

1. Create a vector `engagement` with the values 4, 5, 3, 5, 2 (imagine these are study-engagement ratings).
2. Then add `1` to every value at once.
3. Multiply every value by `2`.

---

9 Picking out values (indexing)

Often you want just one value, or a few, out of a vector. You do this with **square brackets `[ ]`** and a position number. **R counts from 1**, so the first element is position 1.

```
# the first score
scores[1]
#> [1] 28

# the third score
scores[3]
#> [1] 31
```

Pick several at once by putting a vector of positions inside:

```
# the first and second scores
scores[c(1, 2)]
#> [1] 28 35
```

A **minus** sign means "everything except":

```
# all scores except the first
scores[-1]
#> [1] 35 31 40 22
```

Most powerful of all: put a **condition** inside the brackets to keep only the values that match. This is how you **filter** data.

```
# TRUE/FALSE for each score
scores > 30
#> [1] FALSE  TRUE  TRUE  TRUE FALSE

# keep only the scores above 30
scores[scores > 30]
#> [1] 35 31 40
```

Read the last line as: "give me the elements of `scores` where `scores` is greater than 30."

> Going further (optional): combining conditions
>
> Combine conditions with `&` ("and") or `|` ("or"), e.g. `scores[scores > 25 & scores < 40]`.

Your turn

1. From `scores`, print the last value using its position number.
2. Keep only the scores below 30.

(Hint for 1: `scores` has 5 values.)

---

10 Functions

A **function** is a ready-made command. It takes an input, does something with it, and gives back an output. You do not see the work inside; you only give the input and read the output.

A function has the form `function_name(argument)`: you write the name of the function, and put the input in round brackets `( )`. The input in the brackets is called an **argument**. Many useful functions take a vector and summarise it:

```
# how many values?
length(scores)
#> [1] 5

# add them all up
sum(scores)
#> [1] 156

# the average
mean(scores)
#> [1] 31.2

# the smallest
min(scores)
#> [1] 22

# the largest
max(scores)
#> [1] 40
```

Some functions need more than one argument. You separate the arguments with a comma:

```
# round 3.14159 to two decimal places
round(3.14159, 2)
#> [1] 3.14
```

Your turn

Use the `engagement` vector you made earlier. Work out its average yourself: divide its `sum()` by its `length()`. Then compare your answer with `mean(engagement)`. You should get the same number.

---

11 Borrowing extra tools: packages

Base R already does a lot. **Packages** add extra ready-made tools written by others. There are two steps here, and people often confuse them, so read carefully:

- **Install** a package: download it to your computer. You do this **once**.
- **Load** a package with `library()`: switch it on. You do this **every session** you want to use it.

> **Analogy**
>
> Installing a package is like buying a new set of kitchen tools (**functions**) once and putting it in your cupboard. `library()` is taking that set out of the cupboard each time you start cooking. Buying it again every day would be silly; so is re-installing.

The two lines look like this. You try them yourself in a moment:

```
# download once (note: name in quotes)
install.packages("psych")

# switch on each session (no quotes here)
library(psych)
```

> **Watch out**
>
> `install.packages()` needs the name **in quotes**; `library()` does **not**. And if you ever see `Error: there is no package called ...`, it means the package was never installed.

> **Watch out: messages during installing**
>
> While a package installs, the *Console* shows many lines of text, sometimes in red. Red text is **not** always an error: R also uses red for normal messages. Only when a line starts with `Error` did something go wrong. If R asks "Do you want to install from sources…?", type `no` and press *Enter*.

Your turn

Install and load the package `psych`, and use one of its functions.

1. **Install** (once): type `install.packages("psych")` in your script and run it. This takes a moment and needs an internet connection. Wait until the `>` symbol is back in the *Console*.
2. You only need to install once. So put a `#` in front of the line: `# install.packages("psych")`. Now it is a comment, and R skips it the next time you run your script.
3. **Load** (every session): type `library(psych)` on the next line and run it.
4. **Use** a function from the package: run `describe(scores)`. The function `describe()` gives a short summary of a vector in one line.
5. Find the column `mean` in the output. Is it the same number as `mean(scores)`?

Then answer: which of the two lines has quotes around `psych`, and which does not?

---

12 Saving your work

You saved your script as `week1.R` at the start. RStudio does **not** save your changes automatically. When the name on the tab at the top of the *Editor* turns red and gets a `*`, there are changes that are not saved yet.

Save often: press *Ctrl* + *S* on Windows (*Cmd* + *S* on Mac), or click *File* > *Save*.

Your turn

Save your script now. Check that the name on the tab is no longer red. Then open your `research_methods` folder in your file explorer and check that `week1.R` is there.

---

13 Recap: what you can now do

You started today with no programming experience. You can now:

- open RStudio and find your way around the four panes;
- set the working directory;
- create, write in, and save an R script;
- use R as a calculator and write comments;
- store values in objects and follow R's naming rules;
- compare values with operators and read `TRUE` / `FALSE` answers;
- build vectors with `c()` and do maths on all values at once;

- pick out values with `[ ]`, including by a condition;
- use functions such as `mean()` and `length()`;
- explain the difference between installing and loading a package.
- check your working directory with `getwd()`;
- compare values with comparison operators and read `TRUE` / `FALSE` answers;
- install a package, load it with `library()`, and use one of its functions.

A small cheat sheet to keep:

You want to…	You write…
Run a line	*Ctrl* + *Enter* (Mac: *Cmd* + *Enter*)
Check the working directory	`getwd()`
Round a number	`round(x, 2)`
Store a value	`x <- 5`
Make a vector	`v <- c(1, 2, 3)`
Average of a vector	`mean(v)`
Number of elements	`length(v)`
Compare two values	`a == b`, `a > b`
Get the 2nd element	`v[2]`
Keep elements above 3	`v[v > 3]`
Add a comment	`# like this`
Install a package (once)	`install.packages("name")`
Load a package (session)	`library(name)`
Summary from `psych`	`describe(v)`

Well done. Keep your script and your `research_methods` folder – we build directly on them next week.
